Lab 15 — Hands-on lab: a database that outlives its Pods

Goal: request a volume and watch the cluster manufacture it, write into it from two successive Pods, then deploy PostgreSQL as a StatefulSet with its headless Service, delete the Pod, delete the StatefulSet, scale to zero — and get the data back every time.

Prerequisites — Lab 14 completed: k3s; the `postgres:16-alpine` image will be pulled by the cluster from Docker Hub.

Files provided (`files/`) - `pvc-demo.yaml` — a 100 Mi PVC and a `writer` Pod that writes into it. - `secret-db.yaml` — the database password (lab 14). - `statefulset-db.yaml` — the `db` headless Service and the PostgreSQL StatefulSet with `volumeClaimTemplates`.

Step 1 — The StorageClass and a first PVC

```
mkdir -p ~/labo-docker/15 && cd ~/labo-docker/15 && cp <path-to-the-lab>/files/* .
kubectl create namespace lab15 ; kubectl config set-context --current --namespace=lab15
kubectl get storageclass
kubectl apply -f pvc-demo.yaml
sleep 3 ; kubectl get pvc
sleep 10 ; kubectl get pvc ; kubectl get pv
```

Observe `local-path (default) rancher.io/local-path Delete WaitForFirstConsumer` ; a `demo-data` PVC `Pending` if you are quick, then `Bound` to a `pvc-<uuid>` volume of `100Mi RWO` ; and a PV of the same name, `RECLAIM POLICY Delete` , `CLAIM lab15/demo-data` .

```
PV=$(kubectl get pvc demo-data -o jsonpath='{.spec.volumeName}')
kubectl get pv $PV -o jsonpath='{.spec.local.path}{"\n"}
{.spec.nodeAffinity.required.nodeSelectorTerms[0].matchExpressions[0].values[0]}{"\n"}'
kubectl exec writer -- cat /data/journal.txt
sudo ls /var/lib/rancher/k3s/storage/
```

Observe the path `/var/lib/rancher/k3s/storage/pvc-<uuid>_lab15_demo-data` , the affinity to your node, a line `ecrit par writer le …` in the file, and the volume's directory on the node's disk.

Explanation. You asked for 100 Mi; the StorageClass waited for the first Pod (`WaitForFirstConsumer`), then created a directory on the node and a PV bound to your PVC. The Pod only knows the PVC.

Step 2 — The volume outlives the Pod, not the PVC

```
kubectl delete pod writer ; kubectl apply -f pvc-demo.yaml ; sleep 8
kubectl exec writer -- cat /data/journal.txt
kubectl delete pod writer ; kubectl delete pvc demo-data ; sleep 5 ; kubectl get pv | grep -c
demo-data
```

Observe `persistentvolumeclaim/demo-data unchanged` and a new `writer` that reads **two** lines: its own and the previous Pod's; then, after deleting the PVC, `0`: the PV (policy `Delete`) is gone with the data.

Explanation. The PVC holds the volume; Pods come and go. Deleting the PVC triggers the reclaim policy.

Step 3 — PostgreSQL as a StatefulSet

```
kubectl apply -f secret-db.yaml -f statefulset-db.yaml
kubectl rollout status statefulset/db --timeout=120s
kubectl get statefulset,pods,pvc,svc
```

Observe (a `spec.SessionAffinity is ignored for headless services` warning is harmless): `statefulset.apps/db 1/1`, a Pod named **`db-0`** — no hash —, a **`data-db-0`** PVC of `500Mi` bound, and a `db` Service with `CLUSTER-IP None`.

```
kubectl run client --image=docker.io/library/alpine --restart=Never --command -- sleep 600 ;
sleep 6
kubectl exec client -- getent hosts db db-0.db
kubectl exec client -- sh -c 'nc -zv db-0.db.lab15.svc.cluster.local 5432'
kubectl delete pod client --now
```

Observe the same IP under two names, `db.lab15.svc.cluster.local` and **`db-0.db.lab15.svc.cluster.local`**, and port 5432 open: the headless Service gives every Pod a name.

Explanation. `volumeClaimTemplates` created `data-db-0` for `db-0`; the `clusterIP: None` Service does not distribute, it names.

Step 4 — Break the Pod, keep the database

```
kubectl exec db-0 -- psql -U postgres -c "CREATE TABLE clients(id serial, nom text); INSERT
INTO clients(nom) VALUES ('Durand'),('Peeters');"
kubectl delete pod db-0 ; sleep 2 ; kubectl get pods
kubectl rollout status statefulset/db --timeout=120s
kubectl get pvc
kubectl exec db-0 -- psql -U postgres -c "SELECT * FROM clients;"
```

Observe `CREATE TABLE` / `INSERT 0 2`; after the deletion, a Pod called **`db-0`** **again** (`AGE 2s`); the same `data-db-0` PVC; and the two clients found again.

```
kubectl scale statefulset db --replicas=0 ; sleep 10 ; kubectl get pods ; kubectl get pvc
kubectl scale statefulset db --replicas=1 ; kubectl rollout status statefulset/db --
timeout=120s
kubectl exec db-0 -- psql -U postgres -c "SELECT count(*) FROM clients;"
```

Observe `No resources found` for the Pods but `data-db-0` still `Bound` at zero replicas, and `count = 2` on return.

Explanation. The StatefulSet reattaches the `db-0` Pod to the `data-db-0` PVC at every recreation; scaling to zero does not release the volume.

Step 5 — Back up, delete the StatefulSet, recreate

```
kubectl exec db-0 -- pg_dump -U postgres > dump.sql ; grep -A3 'COPY public.clients' dump.sql
kubectl delete statefulset db ; sleep 8 ; kubectl get pods ; kubectl get pvc
kubectl apply -f statefulset-db.yaml ; kubectl rollout status statefulset/db --timeout=120s
kubectl exec db-0 -- psql -U postgres -c "SELECT count(*) FROM clients;"
kubectl get pv
```

Observe the `dump.sql` with the two rows (a logical backup, taken live, as in lab 06); after `delete statefulset`, no Pod but `data-db-0` **still there**; after recreation, `count = 2`; and a PV `Bound` to `lab15/data-db-0`.

Explanation. Deleting a StatefulSet never deletes its PVCs: the database waits for the next StatefulSet of the same name.

Clean-up

```
kubectl delete namespace lab15 ; sleep 20
kubectl get pv | grep -c lab15
kubectl config set-context --current --namespace=default
```

Observe `0`: deleting the namespace deleted the PVC, hence the PV and the data (policy `Delete`). It is the only action in this lab that really destroys the database. Keep `dump.sql`: it is your backup.

What you must be able to state now

- `local-path`, `WaitForFirstConsumer`, `Delete`: a PVC `Pending` until the first Pod, a `pvc-<uuid>` PV in `/var/lib/rancher/k3s/storage/`, with node affinity.
- Two successive Pods read the same `journal.txt`; deleting the PVC deletes the PV.
- StatefulSet: `db-0`, `data-db-0`, Service `CLUSTER-IP None`, `db-0.db.lab15.svc.cluster.local`.
- Pod deleted, scaled to 0, StatefulSet deleted: `data-db-0` stays, `count = 2` on every return.
- Namespace deleted: PV and data gone; `pg_dump` through `kubectl exec` is the backup.